Locality-based moral planning with linear temporal logic values

ADRIAN SATJA KURDIJA*, Faculty of Electrical Engineering and Computing, University of Zagreb, Croatia
MATEJ MIHELČIĆ, Department of Mathematics, Faculty of Science, University of Zagreb, Croatia and Department of Electronics, Ruđer Bošković Institute, Croatia

Linear Temporal Logic (LTL) enables specifying expressive, complex constraints over sequences of actions. A framework for logic-based ethical planning with intended application to robotics was introduced by Grandi et al. in 2022. There, several planning tasks have been shown to be intractable (PSPACE-complete). Using additional assumptions at the cost of generality, we define a constrained version of the CONFLICT problem that is polynomially solvable and present the algorithm that solves it. Motivated from various real-world examples such as robots operating in linear trajectories, we assume that most propositions are *localized* in the following sense: there is an ordering of the localized propositions such that each action depends on, and changes, only the propositions from an interval of bounded length. In addition, there is a constant number of *global* propositions for which there are no such constraints: any action can use or change their truth values. We additionally assume that the local ordering of propositions constrains the temporal order of actions. Under these assumptions, CONFLICT can be solved in polynomial time. The proposed framework can be instrumental in cases were problems require complex spatial robot movement on linear trajectories, especially if work is performed in the presence of humans. Potential collision of a robot with a human could cause serious injury or fatality. Thus, both expressiveness of constraints, affecting choices of the robot's future actions, and speed of their evaluation are crucial to provide proper reactions in unpredictable working environment. A benchmark-based empirical evaluation shows that the proposed locality-based method is substantially faster than the considered baselines on structured locality instances.

JAIR Associate Editor: Insert JAIR AE Name

JAIR Reference Format:
Adrian Satja Kurdija and Matej Mihelčić. 2025. Locality-based moral planning with linear temporal logic values. *Journal of Artificial Intelligence Research* 4, Article 6 (August 2025), 15 pages. DOI: 10.1613/jair.1.xxxxx

1 Introduction

Planning is crucial for autonomous systems like robotics. A framework for logic-based ethical planning with intended application to robotics was introduced by Grandi et al. (Grandi, Lorini, Parker, and Alami 2022). For a key problem, which is to ensure that actions of an agent align with multiple predefined (moral or other) values, linear temporal logic is used to represent values and determine feasible action plans. Evaluation criteria for individual plans were proposed in (Grandi, Lorini, and Parker 2023). General propositional planning problems (ie, PLANSAT) are known to be PSPACE-complete (Bylander 1994), as well as the problem of moral conflict as defined in Grandi et al. (Grandi, Lorini, Parker, and Alami 2022). In this work we introduce locality-based constraints of actions and propositions to reduce the search space and computational complexity.

*Corresponding Author.

Authors' Contact Information: Adrian Satja Kurdija, ORCID: 0000-0003-2313-0396, adrian.kurdija@fer.unizg.hr, Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb, Croatia; Matej Mihelčić, ORCID: 0000-0002-1023-8413, matmih@math.hr, Department of Mathematics, Faculty of Science, University of Zagreb , Zagreb, Croatia, matej.mihelcic@irb.hr and Department of Electronics, Ruđer Bošković Institute , Zagreb, Croatia.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s).
DOI: 10.1613/jair.1.xxxxx

Linear Temporal Logic (LTL) enables specifying constraints over sequences of actions (Huth and Ryan 2004). A set of moral values Ω consists of LTL formulas (Pnueli 1977) that describe temporal properties of atomic propositions (evaluated under some history H) using operators such as negation ($\neg\phi$), conjuction ($\phi_1 \wedge \phi_2$), "next" ($X\phi$), "until" ($\phi_1 \mathrm{U} \phi_2$), "henceforth" ($G\phi$), and "eventually" ($F\phi$). For a given moral problem consiting of moral values Ω, action theory γ, and an initial state s_0, the CONFLICT problem asks if there is an action plan π satisfiying all values from Ω. If there is no such action plan, some values have to be broken, so tradeoffs between values have to be accounted for, which explains the name *moral conflict*. However, we note that the concerned values do not have to be ethical/moral in the strict sense of the word. We decided to follow the previous terminology while keeping in mind the generality of the approach.

Grandi et al. (Grandi, Lorini, Parker, and Alami 2022) have shown that several planning tasks are intractable (PSPACE-complete). Namely, CONFLICT is shown to be PSPACE-complete using reduction from the propositional STRIPS planning problem called PLANSAT (Bylander 1994). The difficulty of the problem arises from the exponential number of possible plans. Using additional assumptions at the cost of generality, we define a constrained version of the CONFLICT problem and present the algorithm to solve it. Namely, to enable polynomial complexity, we restrict actions to affect only a local set of propositions. To complement the theoretical analysis, Section 5 reports an empirical comparison against brute-force and automata/progression baselines.

Such restricted, but computationally efficient setting allows modelling a plethora of real-world scenarios involving planning. One example includes utilizing propositions to express properties involving linear space order. Many robots involved in repairs or manufacturing adhere to such constraints (e.g. robots used to pick and pack food in the factories, gantry robots used in automotive industry, some robots dispatching pallets in the warehouses, in electronic chip manufacturing or pharmaceutical industry in drug production). Utilizing proposed logic, these robots could move back and forth satisfying expressive, complex constraints. One important constraint is not to harm human workers that might cross the path of the traversing robots. This shows the importance of having the possibility to add moral constraints in the planning process to create safe, flexible and efficient robots.

The paper is organized as follows. Sect. 2 describes the related work. In Sect. 3 we describe the framework and the proposed algorithm. In Sect. 4 we present examples. Sect. 5 reports benchmark-based evaluation. Conclusions are given in Sect. 6.

2 Related Work

Logic-based temporal planning has previously been studied in the context of robotics and autonomous systems. Various approaches have been used to address uncertainty, efficiency, and fairness in decision-making. Significant focus has been on using LTL, as well as probabilistic temporal logic (Konur 2011), to define and schedule complex tasks in dynamic and uncertain environments. Earlier work such as (Fainekos et al. 2009) investigated temporal logic motion planning for dynamic robots, formulating a hierarchical approach that integrates motion tracking and formal logic specifications to satisfy high-level tasks. Research (M. Cai, Zhou, et al. 2024) explored methods based on reinforcement learning (RL) to satisfy soft temporal logic constraints in environments where full compliance is not feasible due to motion uncertainties and environmental uncertainties. Some works have investigated probabilistic and reactive planning frameworks. Yoo et al. (Yoo et al. 2013) introduced a probabilistic computation tree logic (PCTL) extension to incorporate resource constraints in motion planning, while Cai et al. (M. Cai, Peng, et al. 2021) developed a probabilistic LTL motion planning strategy that takes into account both environmental uncertainties and motion uncertainties. In contrast, Zhou et al. (Zhou et al. 2025) proposed local observation-based reactive temporal logic planning, focusing on real-time human-robot interactions.

A related research area has been automated reasoning and decision-making for autonomous systems. Meli et al. (Meli et al. 2023) have employed logic programming to improve deliberative robotic task planning, allowing

robots to efficiently evaluate and execute high-level actions. Meanwhile, Shea-Blymyer and Abbas (Shea-Blymyer and Abbas 2021) have formalized *algorithmic ethics* for autonomous systems, introducing a framework based on deontic logic (Von Wright 1951) to satisfy regulatory and ethical constraints. Similarly, Liang and Vasile (Liang and Vasile 2022) addressed concerns about fairness in Mobility-on-Demand systems by using fairness metrics based on temporal logic in decision-making algorithms. In recent work, the use of automata-based approaches has appeared, such as Toscano-Moreno et al. (Toscano-Moreno et al. 2024) which proposed SPIN-based LTL path planning, utilizing model checking for efficient motion planning in environments with uneven terrain. Cai, Liu, Zhou, Ma, Zhao, Wu and Ma (T. Cai et al. 2024) proposed retrieval-augmented reasoning with Large Language Models (LLMs) to improve the interpretability of autonomous vehicle decision making.

A particularly relevant line of work for finite-trace temporal planning is automata/product-state search for LTL_f formulas (Camacho et al. 2018; De Giacomo and Vardi 2013). Intuitively, the planner keeps two pieces of information at each step: the current world state, and a compact summary of "what still needs to happen" for the temporal goals to be satisfied. After every action, this temporal summary is updated; this update is often called progression. Search is then performed over pairs of the form (world state, temporal summary), so two identical world states are treated differently if they have different remaining temporal obligations. The method is general and theoretically clean, but the number of such pairs can grow quickly. Our automata/progression baseline in Section 5 follows this paradigm, while our main method additionally exploits interval locality to keep the explored state space smaller.

3 Locality-Based Planning

Section 3.1 describes the relevant part of the framework defined in (Grandi, Lorini, Parker, and Alami 2022), Section 3.2 describes our additional assumptions and Section 3.3 describes the proposed algorithm.

3.1 Logic-Based Ethical Planning

The world is described by atomic propositions $Prop = \{p_1, \ldots, p_n\}$. Their Boolean truth values are represented as a state $s \in 2^{Prop}$ which consists of all propositions that are true at a given moment.

Linear Temporal Logic over Finite Traces, noted as LTL_f, is a temporal logic used to represent an agent's values and goals over finite traces. LTL_f includes the operators from classical linear temporal logic such as "next" (X) and "until" (U), allowing for expressions about the progression of time and the satisfaction of certain conditions. Additionally, the standard operators "henceforth" (G) and "eventually" (F) are defined as $G\varphi := \neg(\top U \varphi)$ and $F\varphi := \neg G \neg \varphi$. Formally, the grammar of LTL_f is defined as follows ($p \in Prop$):

$$\varphi ::= p \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2 \mid G\varphi \mid F\varphi. \tag{1}$$

In this paper, we expand the language with additional logical operators in LTL_f, defined as follows:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi), \quad \varphi \rightarrow \psi \equiv \neg\varphi \vee \psi. \tag{2}$$

There is a set Act of *actions* and each action $a \in Act$ can change values of some propositions according to the corresponding preconditions. Formally, we define *action theory* as a pair of functions

$$\gamma^+, \gamma^- : Act \times Prop \rightarrow \mathcal{L}_{PL}, \tag{3}$$

where $\mathcal{L}_{PL}$ is the propositional logic fragment of LTL_f. These functions represent preconditions for the actions. Namely, after action a, proposition p becomes true if the precondition $\gamma^+(a, p)$ is true, and false if the precondition $\gamma^-(a, p)$ is true. However, if neither or both preconditions are true, the truth value of p will not change after action a. These rules will be formalized in Eq. (4).

A sequence of actions $\pi = (a_1, \ldots, a_k)$ is called *action plan*.

The described rules of action theory allow us to define a *history* $H(\pi, s_0, \gamma)$ generated by an action plan π from an initial state s_0 under action theory γ. Namely, $H(\pi, s_0, \gamma)$ is a sequence of states $H_0, H_1, H_2, \ldots, H_k \in 2^{Prop}$ with $H_0 = s_0$. To define how H_t transforms into H_{t+1}, we first define the $\models$ relation: we say that $H, t \models \varphi$ if the formula $\varphi \in LTL_f$ holds at time point t in the history H. More specifically,

- $H, t \models p \iff p \in H_t$ for atomic propositions $p \in Prop$.
- $H, t \models \neg\varphi \iff H, t \not\models \varphi$.
- $H, t \models \varphi_1 \wedge \varphi_2 \iff H, t \models \varphi_1$ and $H, t \models \varphi_2$.
- $H, t \models X\varphi \iff H, t+1 \models \varphi$ (ie, φ holds in the next state).
- $H, t \models \varphi_1 U \varphi_2 \iff$ there exists a time $t' \geq t$ such that $H, t' \models \varphi_2$ and for all t'' with $t \leq t'' < t'$ it holds that $H, t'' \models \varphi_1$ (ie, φ_1 holds until φ_2 becomes true).
- $H, t \models F\varphi \iff$ there exists a time $t' \geq t$ such that $H, t' \models \varphi$ (ie, φ holds at some future point, *eventually*).
- $H, t \models G\varphi \iff$ for all $t' \geq t$ it holds that $H, t' \models \varphi$ (ie, φ holds at every point in the future, *henceforth*).

Now, for each action $a_t \in \pi$ and corresponding time t, the next state H_{t+1} is determined according to the action theory γ by updating the propositions in H_t based on the effects of the action a_t:

$$H_{t+1} = \Big(H_t \setminus \{p \in Prop : H_t \models \gamma^-(a_t, p)\}\Big) \cup \{p \in Prop : H_t \models \gamma^+(a_t, p)\}. \tag{4}$$

Here, for each proposition p, we keep or change its truth value based on $\gamma^+(a_t, p)$ and $\gamma^-(a_t, p)$, the positive and negative effect preconditions associated with action a_t.

A set of moral values is given by $\Omega \subseteq LTL_f$. We say that a given moral problem, defined as a triple $\mathcal{M} = (\Omega, \gamma, s_0)$, is a moral conflict if there is no satisfactory action plan. Formaly, $\mathcal{M} \in CONFLICT$ if there is no action plan π such that the history $H(\pi, s_0, \gamma)$ satisfies all values from Ω. Formally, an action plan π is satisfactory if

$$\forall \varphi \in \Omega, \;\; H(\pi, s_0, \gamma), 0 \models \varphi. \tag{5}$$

For example,

- $H, 0 \models p$ when $p \in s_0$,
- $H, 0 \models Fp$ when p is *eventually* satisfied,
- $H, 0 \models Gp$ when p is *always* satisfied,
- $H, 0 \models FGp$ when p is *ultimately* satisfied, ie, it holds in the last state.

If there is no such action plan, some values have to be broken, so tradeoffs between values have to be accounted for, which explains the name *moral conflict*.

Grandi et al. (Grandi, Lorini, Parker, and Alami 2022) showed that CONFLICT is PSPACE-complete using reduction from the propositional STRIPS planning problem called PLANSAT (Bylander 1994). This means that in the general case, the space of all action plans cannot be efficiently searched in order to find a plan satisfying Ω.

3.2 Locality-Based Moral Conflict

Our contribution lies in restricting CONFLICT to a polynomially solvable variant using additional assumptions that reduce its generality without sacrificing too much of its practical use. Motivated from various real-world examples concerning planning in what is essentially a 1D or 2D space (roads, warehouses), such as the example in Sect. 4.1, we assume that most propositions – all except for a constant number of them – are localized. Formaly, we introduce locality as the mapping

$$loc : Prop \to \{0, 1, \ldots, n\}, \tag{6}$$

where locations $1, \ldots, n$ represent positions of the *local* propositions ($loc(p) > 0$), while the propositions with $loc(p) = 0$ are considered *global*. To limit the search space, we posit that there are at most M global propositions

and at most M propositions at each position:

$$\forall i \in \{0, 1, \ldots, n\}, \quad |p \in Prop \ : \ loc(p) = i| \leq M. \tag{7}$$

Each action a depends on, and changes, only the propositions from an interval of at most L locations, along with any global propositions. This local interval depends on the action. To formalize this constraint, we first define $\mathcal{L}_{PL}^{i,j}$ as a fragment of $\mathcal{L}_{PL}$ containing only propositions $p \in Prop$ such that $loc(p) = 0$ or $i \leq loc(p) \leq j$. Now, for each action $a \in Act$, we assume that there are natural constants $l(a)$ and $r(a)$, representing the left and right action boundaries, such that:

$$\begin{aligned} 1 \leq l(a) \leq r(a) \leq n, \\ r(a) - l(a) < L, \\ \gamma^+(a, p_i),\ \gamma^-(a, p_i) \in \mathcal{L}_{PL}^{l(a),r(a)}, \quad \forall i \in [l(a), r(a)], \\ \gamma^+(a, p_i) = \gamma^-(a, p_i) = \bot, \quad \forall i \in [1, n] \setminus [l(a), r(a)]. \end{aligned} \tag{8}$$

Therefore, only the propositions from $\mathcal{L}_{PL}^{l(a),r(a)}$ can be changed by or participate in the preconditions for action a.

Additionally, we assume that the local ordering of propositions constrains the temporal order of actions. Informally, the temporal order of actions is roughly left-to-right with allowed deviations. If the actions' intervals are sufficiently apart, the action on the first interval must be performed before the action on the second. Formally, if $r(b) - l(a) \geq 2L$, then action a cannot be performed after action b. A larger value of L permits more flexibility in the order of actions. In terms of the action plan $\pi = (a_1, \ldots, a_k)$ this condition can be written as

$$r(a_j) - l(a_i) \geq 2L \implies i < j. \tag{9}$$

The final constraint is that the value set Ω contains only formulas with at most one of the operators F, G, FG, or U, operating on a propositional logic formula containing at most K atomic propositions, either all local, or all global. In the local case, the absolute difference between their indices can be at most L.

The integers K, L, M are introduced as fixed constants to limit the search space, and therefore the algorithm complexity. As a reminder, K is the maximum number of propositions in each value formula, L is the maximum number of local propositions each action can influence, and M is the maximum number of global propositions and propositions at each location. For simplicity of complexity analysis, we will assume that $K, L = O(M)$.

3.3 Proposed Algorithm

Under the above assumptions, CONFLICT can be solved in polynomial time. The basic idea stems from the temporal constraint:

$$r(b) - l(a) \geq 2L \implies a \text{ precedes } b,$$

which means that if action a uses/changes p_i, this action precedes all actions b such that $r(b) \geq i + 2L$. Thus, while performing an action plan, we do not need to keep track of "distant" propositions. We only need to keep track of the truth values of the current $2L$-lenght interval of interval of local propositions (called *mutable*) and move it left-to-right, along with the global propositions. The algorithm searches for a path over these substates and the number of such substates is linear in n.

To be able to check the validity of formulas with several local propositions, we must additionally keep track of *recent* local propositions (for which $loc(p) \in \langle i-L, i]$) to the left of the currently *mutable* interval $[i+1, i+2L]$. The truth value of the recent propositions can no longer be changed, but can play a role in validating (or invalidating) formulas where at least one proposition does belong to the mutable interval.

Therefore, the *substate* should contain a truth assignment of an interval of local propositions (from $2L$ mutable and L recent locations) and all global propositions. These propositions of a substate are called *active*. There is

at most $(3L+1)M$ active propositions in a given substate. Additionally, to keep track of temporal values (ie, until and eventually), the substate also contains the set of already satisfied such values composed of the active propositions. Let us formalize all of the above notions.

Definition 1. *A* **substate** *is a triple $S = (i, T, \Theta)$ defined by:*

- *the leading index $i \in [0, n-2L]$ of the mutable interval of local propositions,*
- *the set T of currently true active propositions, where active means global, mutable, or recent: $loc(p) = 0$, $loc(p) \in [i+1, i+2L]$, or $loc(p) \in [\max(1, i-L+1), i]$,*
- *the set $\Theta \subseteq \Omega$ of already satisfied formulas of the form $\psi_1 U \psi_2$ or $F\psi$, where ψ, ψ_1, ψ_2 are propositional logic formulas containing only active propositions.*

By multiplying the number of possible choices for each component of S, the number of substates is $O(n)$ (for the choice of the leading index) with the large constant factor for the choice of other components, dominated by $O(2^{2^M})$ which is the number of formulas in Ω containing only active propositions (the maximal cardinality of Θ).

Assume we are given a CONFLICT instance $\mathcal{M} = (\Omega, \gamma, s_o)$ with assumptions from the previous subsection. We will say that a *global* proposition q is *valid* in a substate S if, informally, all values in Ω containing q are satisfied.

Definition 2. *A global proposition q ($loc(q) = 0$) is* **valid** *in a substate $S = (i, T, \Theta)$ if the following conditions are true for all propositional logic formulas ψ containing q:*

- $FG\psi \in \Omega \implies T \models \psi$,
- $F\psi \in \Omega \implies F\psi \in \Theta$,
- $\psi_1 U \psi_2 \in \Omega \implies \psi_1 U \psi_2 \in \Theta$.

Using this definition, the validity of values containing global propositions can be tested in the final substate: the validity of $FG\psi$ formulas depends on this final state (T), and for $F\psi$ and $\psi_1 U \psi_2$ formulas we check if they have been satisfied at some previous point (Θ).

On the other hand, the validity of a value containing local propositions is tested when all propositions in the value formula exit the mutable interval, since from that point their truth values cannot be changed. Therefore, we say that a local proposition p is valid in a substate if all values in Ω containing p and its left (recent) propositions are satisfied.

Definition 3. *A local proposition p ($loc(p) > 0$) is* **valid** *in a substate $S = (i, T, \Theta)$ if the same three conditions as above (Def. 2) are true for all propositional logic formulas ψ such that ψ contains p and every other local proposition p' in ψ is recent ($loc(p') \in [i-L+1, i]$).*

To verify other types of values such as $G\psi$, we posit that a substate must not invalidate any such formula.

Definition 4. *A substate $S = (i, T, \Theta)$ is* **invalid** *if there is a propositional logic formula containing only active propositions such that any of the following is true:*

- $G\psi \in \Omega \;\wedge\; T \not\models \psi$,
- $\psi_1 U \psi_2 \in \Omega \;\wedge\; \psi_1 U \psi_2 \notin \Theta \;\wedge\; T \not\models \psi_1 \;\wedge\; T \not\models \psi_2$.

If a substate is not invalid, we say that it is **valid**.

The formula $\psi_1 U \psi_2$ becomes satisfied and enters Θ when $\psi_2 \in T$, assuming that all previous substates were valid (which ensures that ψ_1 was true). The formula $F\psi$ enters Θ when $T \models \psi$.

Definition 5. *Define the* **initial** *substate S_0 as the substate (i, T, Θ) with $i = 0$, $T = s_0 \cap \{p \in Prop : loc(p) \leq 2L\}$, and $\Theta \subseteq \Omega$ is the set of formulas that are satisfied in the initial state s_0, have the form $\psi_1 U \psi_2$ or $F\psi$, and contain only active propositions ($loc(p) \leq 2L$).*

The algorithm that determines if $\mathcal{M}$ is a moral conflict consists in finding an appropriate sequence of substates which simulates a satisfactory action plan. The main idea is to change substates by applying actions that fit the current interval, and move to the next interval when no more actions apply to the left proposition of the interval. Let us formalize the requirements of such a path.

Definition 6. *A directed graph $\mathcal{G}(\mathcal{M})$ (or simply $\mathcal{G}$) has all valid substates as vertices. We define the edges of $\mathcal{G}$ as follows. An edge from substate S to substate S' exists if any of the following two cases ("types" of edges) is satisfied.*

Type 1: *There is an action that changes S to S'. Formally, there is an index i such that $S = (i, T, \Theta)$ and $S' = (i, T', \Theta')$, and there exists $a \in Act$ such that:*

- $i + 1 \leq l(a) \leq r(a) \leq i + 2L$,
- $T' = T \setminus \{p \in Prop : T \models \gamma^-(a, p),\ loc(p) \in [l(a), r(a)] \cup \{0\}\}$
  $\cup \{p \in Prop : T \models \gamma^+(a, p),\ loc(p) \in [l(a), r(a)] \cup \{0\}\}$,
- *Θ' is a union of Θ and the set of all values from Ω satisfied in T' of the form $\psi_1 U \psi_2$ or $F\psi$, containing only active propositions.*

Type 2: *Move right (increase the index). Formally, there is an index i such that $S = (i-1, T, \Theta)$ and $S' = (i, T', \Theta')$, and*

- $T' = T \setminus \{p \in Prop : loc(p) = i - L\} \cup (s_0 \cap \{p \in Prop : loc(p) = i + 2L\})$,
- *Θ' is obtained by removing from Θ the formulas containing propositions p with $loc(p) = i - L$ and adding all values from Ω satisfied in T' of the form $\psi_1 U \psi_2$ or $F\psi$, containing only active propositions,*
- *all propositions such that $loc(p) = i$ are valid in S.*

The last point ensures the validity of the formulas with local propositions that will no longer change their truth value.

Definition 7. *A vertex in $\mathcal{G}$ is* **final** *if it corresponds to a substate $S = (i, T, \Theta)$ such that $i = n - 2L$ (active interval covers the end of Loc) and all propositions with $loc(p) > i$ or $loc(p) = 0$ are valid in S.*

Assume that there is a path $\mathcal{P}$ in $\mathcal{G}$ from the initial state S_0 to a final state T. To generate an action plan π that satisfies Ω, we simply take the corresponding actions from all edges of Type 1 from $\mathcal{P}$ in the same order. Therefore, the algorithm for the locality-based CONFLICT problem is given in Algorithm 1 and the following theorem holds.

Theorem 1. *There is a plan $\pi = (a_1, \ldots, a_k)$ such that $H(\pi, s_0, \gamma)$ satisfies Ω if and only if there is a path in $\mathcal{G}$ from the initial vertex S_0 to a final vertex.*

Proof. Algorithm 1 is a depth-first search for any path in $\mathcal{G}$ from the initial vertex S_0 to a final vertex. Therefore, if a path exists, Algorithm 1 will find it and produce the corresponding action plan π. By construction of $\mathcal{G}$, it is clear that $H(\pi, s_0, \gamma)$ satisfies Ω.

For the other direction, assume that $\pi = (a_1, \ldots, a_k)$ is a satisfying action plan. We can construct the corresponding path in $\mathcal{G}$ (sequence of substates) as follows. Start with $t = 1$ and S_0 as the initial substate. For each substate index $i = 0, 1, \ldots, n - 2L$, perform the following subprocedure. Apply actions in π from a_t onwards (by increasing t) such that $r(a_t) \leq i + 2L$, update the current substate according to the action effects (Type-1 edge), and finally, when the next action no longer satisfies $r(a_t) \leq i + 2L$, move to the next substate index $i + 1$ (Type-2 edge). Condition (9) ensures that after this subprocedure, no subsequent actions will change propositions with $loc(p) = i + 1$, which means they are valid at that point (the action plan satisfies Ω by assumption). For the same reason, all propositions when $i = n - 2L$ must be valid. Thus, the state is final. Figure 1 illustrates one potential path in a graph of substates. □

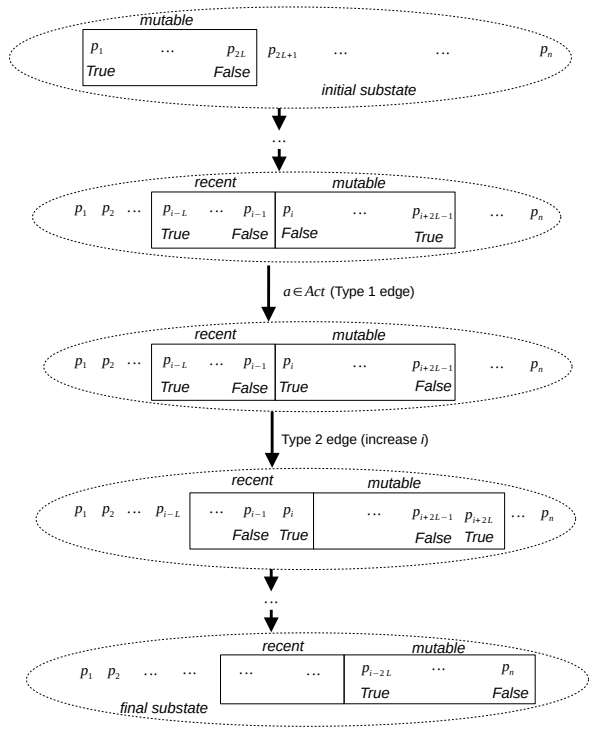

Fig. 1. Illustration of a path in the graph of substates. The index of a proposition represents its location ($l(p_i) = i$)

The computational complexity of the locality-based CONFLICT is: $O(n(|Act| + |\Omega|))$ for graph construction and $O(n^2)$ for a breath-first or depth-first search on a graph of substates. Thus, the locality-based CONFLICT is polynomial.

Because of the large constant factor (simplified to $O(2^{2^M})$ for $K, L = O(M)$), the theoretical strength of this results depends on how small the constants are, which reveals a tradeoff between efficiency and flexibility. However, this worst-case constant will almost never be obtained in practice since the concerned cardinality of Θ (a local subset of Ω) will usually be far less than the number of all possible propositional logic formulas.

4 Example

4.1 Traffic Lights

Consider a system that fixes traffic lights along a road. There are n intersections denoted from 1 to n in the order of their position along the road. Let the set of local propositions $Loc = \bigcup_{i=1}^{n} \{on_i, broken_i\}$ denote whether the traffic lights on the corresponding intersections are turned on and whether they are broken (a traffic light can be turned off without being broken). Let the set of global propositions be $Glob = \{congestion\}$ and let $\Omega = \{FGon_i, i = 1, \ldots, n\} \cup \{G\neg congestion\}$, meaning that we want to ultimately fix and turn on all the traffic lights without ever causing congestion. For each intersection, there are three possible actions: f_i fixes and turns on the traffic light, g_i turns it off, and h_i turns it on if it is not broken. Due to synchronicity issues, we can fix the traffic light only if one of the neighboring traffic lights is off. However, if both of them are off while repairing, it

Algorithm 1 Locality-based CONFLICT

Require: $Prop, loc, Act, \Omega, \gamma, s_0, L$
Ensure: π ▹ Returns a satisfying action plan π or False if it does not exist.

```
push(S_0)                                   ▹ Push the initial substate S_0 to the stack.
while stack is not empty do
    S := pop()                              ▹ Take a substate from the stack.
    if visited(S) or not valid(S) then
        continue;
    end if
    if final(S) then
        π := list()                         ▹ Initialize empty list.
        S_0, S_1, ..., S := stack           ▹ Read stack bottom-up.
        for j = 0, 1, ... do
            if a ∈ Act changes S_j to S_{j+1} then
                π.append(a)
            end if
        end forreturn π
    end if
    visited(S) := True
    for a ∈ Act do
        if a changes S to S' then
            push(S')                        ▹ Type 1 edge.
        end if
    end for
    Construct S' from S as in Def. 6 (Type 2)
    push(S')
end while
return False
```

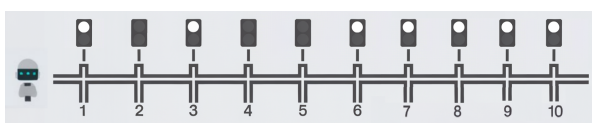

Fig. 2. Example 4.1: Traffic lights 2, 4, and 5 are broken.

causes congestion. Formally,

$$\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = \neg on_{i-1} \vee \neg on_{i+1}, \quad \text{for } 1 < i < n,$$
$$\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = \top \quad \text{for } i \in \{1, n\},$$
$$\gamma^+(f_i, congestion) = \neg on_{i-1} \wedge \neg on_{i+1} \quad \text{for } 1 < i < n,$$
$$\gamma^-(g_i, on_i) = \top, \quad \gamma^+(h_i, on_i) = \neg broken_i, \quad \text{for } 1 \leq i \leq n. \quad (10)$$

$\mathcal{M} = (\Omega, \gamma, s_0)$ is a locality-based moral conflict for $M = |Glob| = 1$ and $L = 3$ since each action concerns at most three consecutive local propositions. $L = 3$ also means that actions f_i, g_i and h_i can be performed after f_j, g_j

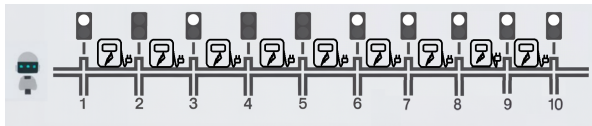

Fig. 3. Example 4.2: Traffic lights 2, 4, and 5 are broken, and there is a charger between each pair of lights. Fixing traffic light 4 drains the robot's battery.

and h_j for $j \leq i+3$, but not not after f_{i+4}, g_{i+4} and h_{i+4}. Thus, the order of fixing traffic lights is roughly similar to their order along the road, which is a natural constraint.

For $n = 10$ and the initial state $s_0 = \{on_1, broken_2, on_3, broken_4, broken_5, on_6, on_7, on_8, on_9, on_{10}\}$ as depicted in Figure 2, $\mathcal{M}$ is not a moral conflict. There is an action plan $\pi = (g_3, f_2, h_3, f_4, g_6, f_5, h_6)$. Namely, g_3 turns off light 3 so we can fix light 2 using f_2; then we turn on light 3 again using h_3. We can then fix light 4 using f_4. If we used f_4 before h_3 (ie, before light 3 was turned on), we would cause congestion. To fix light 5, we turn off light 6 using g_6 and then perform f_5 before turning light 6 on again using h_6. For this example, the proposed algorithm finds the following path, ie, the sequence of substates:

$$
\begin{aligned}
& (0, \{\{on_1, broken_2, on_3, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } g_3 \rightarrow\ & (0, \{\{on_1, broken_2, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } f_2 \rightarrow\ & (0, \{\{on_1, on_2, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } h_3 \rightarrow\ & (0, \{\{on_1, on_2, on_3, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{move } \rightarrow\ & (1, \{\{on_1, on_2, on_3, broken_4, broken_5, on_6, on_7\}, \emptyset\}, \emptyset) \\
\text{perform } f_4 \rightarrow\ & (1, \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7\}, \emptyset\}, \emptyset) \\
\text{move } \rightarrow\ & (2, \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8\}, \emptyset\}, \emptyset) \\
\text{perform } g_6 \rightarrow\ & (2, \{\{on_1, on_2, on_3, on_4, broken_5, on_7, on_8\}, \emptyset\}, \emptyset) \\
\text{perform } f_5 \rightarrow\ & (2, \{\{on_1, on_2, on_3, on_4, on_5, on_7, on_8\}, \emptyset\}, \emptyset) \\
\text{perform } h_6 \rightarrow\ & (2, \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8\}, \emptyset\}, \emptyset) \\
\text{move } \rightarrow\ & (3, \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9\}, \emptyset\}, \emptyset) \\
\text{move } \rightarrow\ & (4, \{\{on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9, on_{10}\}, \emptyset\}, \emptyset).
\end{aligned}
$$

4.2 Traffic Lights with Chargers

As depicted in Figure 3, we expand the previous example by introducing charging stations between each pair of traffic lights. The charging stations allow the robot to recharge its battery. However, we want to avoid wasting battery charging cycles when it is not necessary, so we will disallow charging until the battery runs low due to major repair work. We introduce an action c_i that fully charges the robot at the charging station located in front of the traffic light with the index i. If a proposition $charged_i$ denotes that this charger has been used, the described condition can be written as $\neg charged_i\ U\ lowBattery$.
Formally, we have:

$$
\begin{aligned}
& Loc = \cup_{i=1}^{n} \{on_i,\ broken_i, charged_i\},\ Glob = \{congestion, lowBattery\}, \\
& \Omega = \{FGon_i, i = 1, \ldots, n\}\ \cup \{\neg charged_i\ U\ lowBattery, i = 1, \ldots, n\} \cup \{\ G\ \neg congestion\}\ \cup \{FG\ \neg lowBattery\}.
\end{aligned} \tag{11}
$$

The initial state is the same as in Example 4.1, extended with battery-related propositions: $lowBattery$ is false and $charged_i$ is false for all i. We update the actions so that

$$\gamma^-(c_i, lowBattery) = \gamma^+(c_i, charged_i) = \top. \tag{12}$$

Major repair work will include fixing traffic light 4, thus $\gamma^+(f_4, lowBattery) = \top$. Actions f_i, g_i, h_i can be performed only if the battery is not low:

$$\begin{aligned} \gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = (\neg on_{i-1} \vee \neg on_{i+1}) \ \wedge \neg lowBattery \quad &\text{for } 1 < i < n, \\ \gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = \neg lowBattery \quad &\text{for } i \in \{1, n\}, \\ \gamma^-(g_i, on_i) = \neg lowBattery,\ \gamma^+(h_i, on_i) = \neg broken_i \wedge \neg lowBattery \quad &\text{for } 1 \le i \le n. \end{aligned} \tag{13}$$

Again, since each action affects at most three consecutive local propositions, we have $L = 3$. The algorithm finds the following path, ie, the sequence of substates:

$$\begin{aligned} & (0,\ \{\{on_1, broken_2, on_3, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\ \text{perform } g_3 \rightarrow\ & (0,\ \{\{on_1, broken_2, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\ \text{perform } f_2 \rightarrow\ & (0,\ \{\{on_1, on_2, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\ \text{perform } h_3 \rightarrow\ & (0,\ \{\{on_1, on_2, on_3, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\ \text{move} \rightarrow\ & (1,\ \{\{on_1, on_2, on_3, broken_4, broken_5, on_6, on_7\},\ \emptyset\}, \emptyset) \\ \text{perform } f_4 \rightarrow\ & (1,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7\},\ \{lowBattery\}\}, \cup_{i=1}^{7}\{\neg charged_i\ U\ lowBattery\}) \\ \text{move} \rightarrow\ & (2,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8\},\ \{lowBattery\}\}, \cup_{i=1}^{8}\{\neg charged_i\ U\ lowBattery\}) \\ \text{perform } c_4 \rightarrow\ & (2,\ \{\{on_1, on_2, on_3, on_4, charged_5, broken_5, on_6, on_7, on_8\},\ \emptyset\}, \cup_{i=1}^{8}\{\neg charged_i\ U\ lowBattery\}) \\ \text{move} \rightarrow\ & (3,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8, on_9\},\ \emptyset\}, \cup_{i=1}^{9}\{\neg charged_i\ U\ lowBattery\}) \\ \text{perform } g_6 \rightarrow\ & (3,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_7, on_8, on_9\},\ \emptyset\}, \cup_{i=1}^{9}\{\neg charged_i\ U\ lowBattery\}) \\ \text{perform } f_5 \rightarrow\ & (3,\ \{\{on_1, on_2, on_3, on_4, on_5, on_7, on_8, on_9\},\ \emptyset\}, \cup_{i=1}^{9}\{\neg charged_i\ U\ lowBattery\}) \\ \text{perform } h_6 \rightarrow\ & (3,\ \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9\},\ \emptyset\}, \cup_{i=1}^{9}\{\neg charged_i\ U\ lowBattery\}) \\ \text{move} \rightarrow\ & (4,\ \{\{on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9, on_{10}\},\ \emptyset\}, \cup_{i=2}^{10}\{\neg charged_i\ U\ lowBattery\}) \end{aligned}$$

The last substate is indeed final since $i = n - 2L$ and all active propositions are valid in the sense of Definitions 2 and 3. For example, $FGon_7$ and $FG\neg lowBattery$ are true, while $\neg charged_7\ U\ lowBettery$ does belong to Θ. The validity of previous local propositions, such as on_1, has been verified as part of the "move" operation according to Definition 6, while the truth of $G\neg congestion$ was verified in each substate according to Definition 4.

4.3 Narrow Factory Grid (2D)

The previous examples are effectively one-dimensional: intersections are ordered along a road. We now show a two-dimensional setting that still satisfies the locality assumptions when one dimension is small. Consider a long production line with n columns and W parallel conveyor lanes, where W is a fixed small constant (eg, $W = 3$). The robot can move forward along the line and switch between neighboring lanes. Human workers occasionally cross the lanes, and the robot must avoid collisions.

We index each cell by (i, y) where $i \in \{1, \ldots, n\}$ is the column and $y \in \{1, \ldots, W\}$ is the lane. The key modeling choice is to map locality to columns, ie, we define $loc(\cdot) = i$ for every proposition indexed by (i, y). This keeps the "long" direction explicit, while the lane index is absorbed into the bounded number of propositions per location.

Formally,

$$Loc = \bigcup_{i=1}^{n} \{robot_{i,y}, broken_{i,y}, done_{i,y}, human_{i,y} : y = 1, \ldots, W\}, \qquad Glob = \{collision\}. \tag{14}$$

Hence each location i has at most $4W$ local propositions, so $M = 4W$ is constant when W is fixed.

Let $R \subseteq \{1, \ldots, n\} \times \{1, \ldots, W\}$ be the set of stations that must be serviced. The value set is

$$\Omega = \{G\neg collision\} \ \cup \ \{FG\, done_{i,y} : (i, y) \in R\}. \tag{15}$$

Thus, collisions are always forbidden, and every required station must eventually be repaired and remain repaired.

We use three action families:

- *Forward move* $m_{i,y}$ for $1 \leq i < n$: move from (i, y) to $(i + 1, y)$.
- *Lane change* $s_{i,y\to y'}$ for $|y - y'| = 1$: move at fixed column i from lane y to lane y'.
- *Repair* $rep_{i,y}$: repair station at (i, y).

Representative effects are:

$$\gamma^-(m_{i,y}, robot_{i,y}) = \gamma^+(m_{i,y}, robot_{i+1,y}) = \top, \qquad \gamma^+(m_{i,y}, collision) = human_{i+1,y}, \tag{16}$$

$$\gamma^-(s_{i,y\to y'}, robot_{i,y}) = \gamma^+(s_{i,y\to y'}, robot_{i,y'}) = \top, \qquad \gamma^+(s_{i,y\to y'}, collision) = human_{i,y'}, \tag{17}$$

$$\gamma^-(rep_{i,y}, broken_{i,y}) = \gamma^+(rep_{i,y}, done_{i,y}) = robot_{i,y} \wedge \neg human_{i,y}, \qquad \gamma^+(rep_{i,y}, collision) = robot_{i,y} \wedge human_{i,y}. \tag{18}$$

All other effects are $\bot$. In this model, move actions touch only columns i and $i + 1$, lane changes and repairs touch only column i, and all of them may touch one global proposition ($collision$). Therefore $r(a) - l(a) \leq 1$ for all actions, so the locality requirement is satisfied for $L = 2$.

As a concrete instance, let $n = 12$, $W = 3$, $R = \{(3, 2), (7, 2), (9, 2)\}$, and let the initial robot position be $(1, 2)$. Suppose crossing zones $(4, 2)$ and $(8, 2)$ are occupied by humans. A satisfying plan is:

$$\pi = (m_{1,2}, m_{2,2}, rep_{3,2}, s_{3,2\to 3}, m_{3,3}, m_{4,3}, s_{5,3\to 2}, m_{5,2}, m_{6,2}, rep_{7,2}, s_{7,2\to 1}, m_{7,1}, m_{8,1}, s_{9,1\to 2}, rep_{9,2}).$$

The robot repairs required cells and detours around occupied crossing zones by switching lanes before entering unsafe cells. This example is two-dimensional in geometry, but remains one-dimensional in locality indexing with a fixed-width augmentation, which is exactly the regime targeted by our complexity assumptions.

5 Evaluation

This section evaluates whether the theoretical locality advantage translates into practical runtime gains. We compare three planners under a common protocol: the proposed locality-based planner, a brute-force planner with online value pruning, and an automata/progression LTL_f baseline. The automata/progression baseline follows the standard finite-trace temporal-logic approach in which search is performed on a product between planning states and temporal-monitor states (Camacho et al. 2018; De Giacomo and Vardi 2013); operationally, each value formula is represented by a residual formula that is progressed after each action. In all experiments, every produced plan is checked by an independent validator, so correctness is not inferred from internal planner state.

5.1 Setup

The benchmark set consists of 57 systematically generated instances with problem size $n \in [6, 24]$ and three instances per size. These instances follow the traffic-light locality structure used throughout the paper: actions depend on and modify bounded neighborhoods, while values include global safety constraints and eventual local repair goals. The generation procedure is deterministic and size-ordered, with broken-light configurations chosen to avoid trivial adjacency patterns whenever possible. This design provides controlled growth in instance size while preserving the locality assumptions studied in Section 3.2.

All methods are run with the same maximum search depth (160) and evaluated per instance under fixed runtime limits. We use a stricter limit for the locality planner (10 seconds) and a looser limit for both baselines (30 seconds), so the comparison does not disadvantage the non-local methods. The locality parameter is fixed at $L = 3$, matching the locality structure of the benchmark family. A run is counted as successful only if a planner returns a plan within the time limit and that plan is accepted by independent validation. Reported planning times exclude validation time, so runtime values represent planner-only cost.

5.2 Results

Table 1 summarizes success counts and runtime statistics. The locality planner solves all 57 instances, and the automata/progression baseline also solves all 57 instances. The brute-force baseline solves 47 instances and times out on 10. Runtime statistics are computed over successful runs only: the locality planner has mean runtime 0.001s, median 0.00s, and maximum 0.01s; the automata/progression baseline has mean 1.26s, median 0.01s, and maximum 22.89s; and the brute-force baseline has mean 2.90s, median 0.02s, and maximum 27.66s.

Table 1. Benchmark summary over 57 instances.

Method	Success	Timeout	Mean (s)	Median (s)	Max (s)
Locality-based planner	57/57	0	0.001	0.00	0.01
Automata/progression baseline	57/57	0	1.26	0.01	22.89
Brute-force baseline	47/57	10	2.90	0.02	27.66

5.3 Results Discussion

The central empirical observation is that locality-aware state compression dominates on this benchmark family. Even with the strictest time budget, the locality planner solves every instance and remains in a near-constant time regime across the tested range of n. By contrast, the two baselines exhibit heavier tails in runtime, especially on larger instances. The automata/progression baseline remains complete on this set, but its worst-case runtime is orders of magnitude larger than that of the locality planner. This behavior is expected: progression-based search tracks temporal residuals over a larger effective product space and does not directly exploit interval locality in the same explicit way.

The brute-force baseline shows the weakest scalability. Its median successful runtime is small, indicating that easy instances are handled quickly, but the timeout count demonstrates loss of robustness as size increases. This gap between median and timeout behavior is important: median-only summaries would hide the heavy-tail effect, while the success/timeout statistics reveal practical reliability.

These results are consistent with the theory from Section 3.3. The proposed method keeps only a bounded active window of local propositions (plus global propositions and temporal satisfaction memory) and shifts this window left-to-right. In structured domains that satisfy the locality assumptions, this restriction sharply reduces

branching and effective state size. The baselines rely on more general temporal-monitoring search and therefore pay a larger per-instance overhead.

At the same time, the interpretation should remain scoped to the evaluated regime. The benchmark family is intentionally locality-structured and therefore aligned with the assumptions of our algorithm. This is a strength for validating the intended use case, but it is not evidence that the same margins will hold on domains with weak locality or different temporal-value distributions. A broader study with additional domain families, parameter sweeps over L, and cross-platform timing would provide a stronger estimate of external validity.

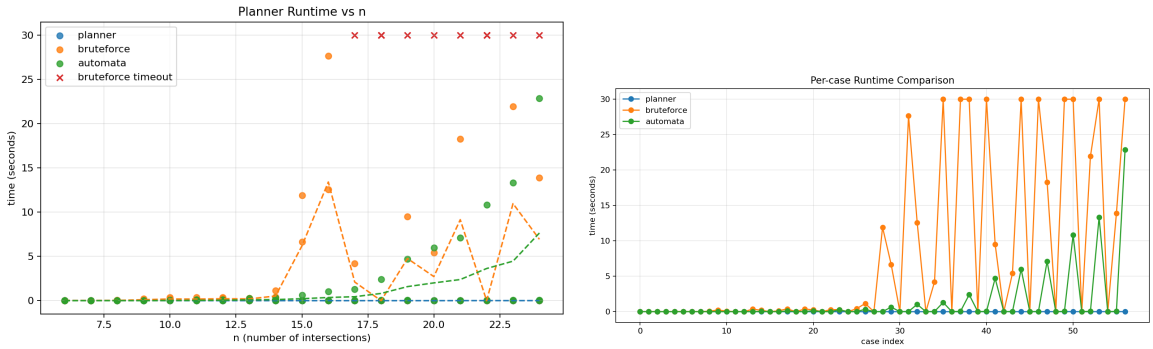

Fig. 4. Benchmark plots: runtime versus instance size n (left) and runtime per case (right).

6 Conclusion

We introduced a locality-based approach to solving the CONFLICT problem in ethical planning using Linear Temporal Logic (LTL). By incorporating assumptions on the structure of the problem, the distinction between global and local propositions and the constraints on the scope and sequence of actions, we were able to reduce the complexity of the problem from PSPACE-complete to polynomial time. The introduced concept of localized planning restricts the actions to only affect and depend on propositions within a bounded interval. It incorporates constraints on the temporal order of actions based on this interval. Due to possibility of large constant factors in the algorithm complexity, there might be a tradeoff between efficiency and flexibility. We consider the proposed algorithm to be practical for real-world applications with small values of constants K, L, and M, ie, the applications with "high locality". The evaluation in Section 5 supports this claim by showing strong runtime and robustness advantages on systematic locality-structured benchmarks.

Overall, our work offers a promising direction for efficient and scalable ethical planning in autonomous systems and robotics. By constraining the problem space in a principled way, we enable the development of action plans that adhere to predefined values, ensuring ethical behavior in complex, dynamic environments. Future research could explore extending this framework to accommodate more complex moral structures and larger sets of global propositions, as well as applying these methods to real-world robotic systems and other automated decision-making contexts.

References

T. Bylander. 1994. "The computational complexity of propositional STRIPS planning." *Artificial Intelligence*, 69, 1, 165–204. doi:https://doi.org/10.1016/0004-3702(94)90081-7.

M. Cai, H. Peng, Z. Li, and Z. Kan. 2021. "Learning-Based Probabilistic LTL Motion Planning With Environment and Motion Uncertainties." *IEEE Transactions on Automatic Control*, 66, 5, 2386–2392. doi:10.1109/TAC.2020.3006967.

M. Cai, Z. Zhou, L. Li, S. Xiao, and Z. Kan. 2024. "Reinforcement learning with soft temporal logic constraints using limit-deterministic generalized Büchi automaton." *Journal of Automation and Intelligence*. doi:https://doi.org/10.1016/j.jai.2024.12.005.

T. Cai, Y. Liu, Z. Zhou, H. Ma, S. Z. Zhao, Z. Wu, and J. Ma. 2024. *Driving with Regulation: Interpretable Decision-Making for Autonomous Vehicles with Retrieval-Augmented Reasoning via LLM*. (2024). https://arxiv.org/abs/2410.04759 arXiv: 2410.04759 (cs.AI).

A. Camacho, J. A. Baier, C. Muise, and S. A. McIlraith. 2018. *Finite LTL Synthesis as Planning*. Automata/product-state perspective for finite-trace planning. (2018).

G. De Giacomo and M. Y. Vardi. 2013. *Linear Temporal Logic and Linear Dynamic Logic on Finite Traces*. Finite-trace temporal logic background. (2013).

G. E. Fainekos, A. Girard, H. Kress-Gazit, and G. J. Pappas. 2009. "Temporal logic motion planning for dynamic robots." *Automatica*, 45, 2, 343–352. doi:https://doi.org/10.1016/j.automatica.2008.08.008.

U. Grandi, E. Lorini, and T. Parker. 2023. "Moral planning agents with ltl values." In: *32nd International Joint Conference on Artificial Intelligence (IJCAI 2023)*. International Joint Conferences on Artificial Intelligence Organization, 418–426.

U. Grandi, E. Lorini, T. Parker, and R. Alami. 2022. "Logic-based ethical planning." In: *International Conference of the Italian Association for Artificial Intelligence*. Springer, 198–211.

M. Huth and M. Ryan. 2004. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, USA. ISBN: 052154310X.

S. Konur. 2011. *Real-time and Probabilistic Temporal Logics: An Overview*. (2011). https://arxiv.org/abs/1005.3200 arXiv: 1005.3200 (cs.LO).

K. Liang and C.-I. Vasile. 2022. "Fair Planning for Mobility-on-Demand with Temporal Logic Requests." In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1283–1289. doi:10.1109/IROS47612.2022.9981291.

D. Meli, H. Nakawala, and P. Fiorini. 2023. "Logic Programming for Deliberative Robotic Task Planning." *Artificial Intelligence Review*, 56, 9011–9049. doi:10.1007/s10462-022-10389-w.

A. Pnueli. 1977. "The temporal logic of programs." In: *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, 46–57. doi:10.1109/SFCS.1977.32.

C. Shea-Blymyer and H. Abbas. Sept. 2021. "Algorithmic Ethics: Formalization and Verification of Autonomous Vehicle Obligations." *ACM Trans. Cyber-Phys. Syst.*, 5, 4, Article 38, (Sept. 2021), 25 pages. doi:10.1145/3460975.

M. Toscano-Moreno, A. Mandow, M. A. Martínez, and A. J. García-Cerezo. 2024. "SPIN-Based Linear Temporal Logic Path Planning for Ground Vehicle Missions with Motion Constraints on Digital Elevation Models." *Sensors*, 24, 16. doi:10.3390/s24165166.

G. H. Von Wright. 1951. "Deontic logic." *Mind*, 60, 237, 1–15.

"Probabilistic Temporal Logic for Motion Planning with Resource Threshold Constraints." (July 2013), 457–464. ISBN: 9780262315722. doi:10.7551/mitpress/9816.003.0063.

Z. Zhou, S. Wang, Z. Chen, M. Cai, H. Wang, Z. Li, and Z. Kan. 2025. "Local Observation Based Reactive Temporal Logic Planning of Human-Robot Systems." *IEEE Transactions on Automation Science and Engineering*, 22, 643–655. doi:10.1109/TASE.2023.3304842.

Received ; accepted